

First Day of Formal Verification Learning!

Installation, vocabularies, troubleshooting, oh my!



SHECRYPTS
AUG 16, 2026

It's been a while!

8/15/2026

8:12 pm

I decided to start this project about two weeks ago, but we moved apartments, so I'm working on my Day 1 material today. This is going to be my first time reading in detail about formal verification and dipping my toes in it. I also have no clue about what to install or how to install them, so I'm gonna read the tutorial.

Thanks for reading shecrypts Publication!
Subscribe for free to receive new posts and
support my work.

✓ **Subscribed**

What I know so far is that formal verification uses *proof assistants* to prove that an algorithm does what it claims to do. To do formal verification, you need to understand logic and mathematical proofs.

My goal today is to be able to install Isabelle, know the basic use and prove a very simple statement. Why Isabelle? Although I don't know

how it works, it was one of the programs listed in the [job description](#) I found! You know what? Let me Google a short description real quick!

Here is what I found! (That was me writing this sentence, not ChatGPT!)

Isabelle is one of the most practical and industry-tested proof assistants for algorithm verification and programming language semantics. Instead of writing cryptic terminal scripts, Isabelle uses a human-readable proof language called **Isar** that reads like a formal textbook proof (eg: fix x, assume P, show Q).

Since I'm used to math textbooks and research papers, learning Isabelle will be easier than learning other formal verification tools like Lean.

While Lean is currently the favorite for abstract pure mathematics, Lean proofs rely heavily on procedural, step-by-step tactic scripts. Also, my goal is to do formal verification for cryptography, and Isabelle has a mature framework for that (CryptHOL). I may decide to learn Lean in the future, but I want to focus on grasping formal verification for now rather than learning a lot of new code. (I guess this is me trying to convince myself that I made the right decision!)

The major learning resources for Isabelle are

- **Prog-Prove**: The official ~100-page quick-start guide available in Isabelle's documentation. It covers basic logic, standard data types, induction, and the simplifier without getting bogged down in theory.
- **Concrete Semantics (Nipkow & Klein)**: The definitive free online textbook. It teaches Isabelle through the lens of programming language semantics, structural operational semantics, and program verification.
- **Archive of Formal Proofs (AFP)**: A massive, peer-reviewed repository of Isabelle theories. Browsing the AFP is the best way to learn how experienced users structure production-grade proofs.

I'm learning Isabelle/HOL. **Isabelle/HOL** stands for the **Isabelle** proof assistant software running the **Higher-Order Logic (HOL)** mathematical system.

Isabelle (The Engine): The underlying software framework that provides the proof environment, term-rewriting engine, user interface, and proof-automation tools like Sledgehammer. (from [Wikipedia](#))

HOL (Higher-Order Logic): The specific logical language loaded into Isabelle. It provides everyday mathematical constructs such as booleans, natural numbers, sets, functions, and typed logic—essentially functioning as a combination of functional programming and formal logic. (from [Prog-Prove](#))

Isabelle can be viewed as a game console framework, and **HOL** is the primary software environment loaded into it. While Isabelle can technically run other logic systems (like First-Order Logic Isabelle/FOL or Set Theory Isabelle/ZF), HOL is the standard distribution used for software verification and formal proofs.

I'm following the tutorials and books step-by-step here and doing multiple Google searches along the way because I'm new to everything.

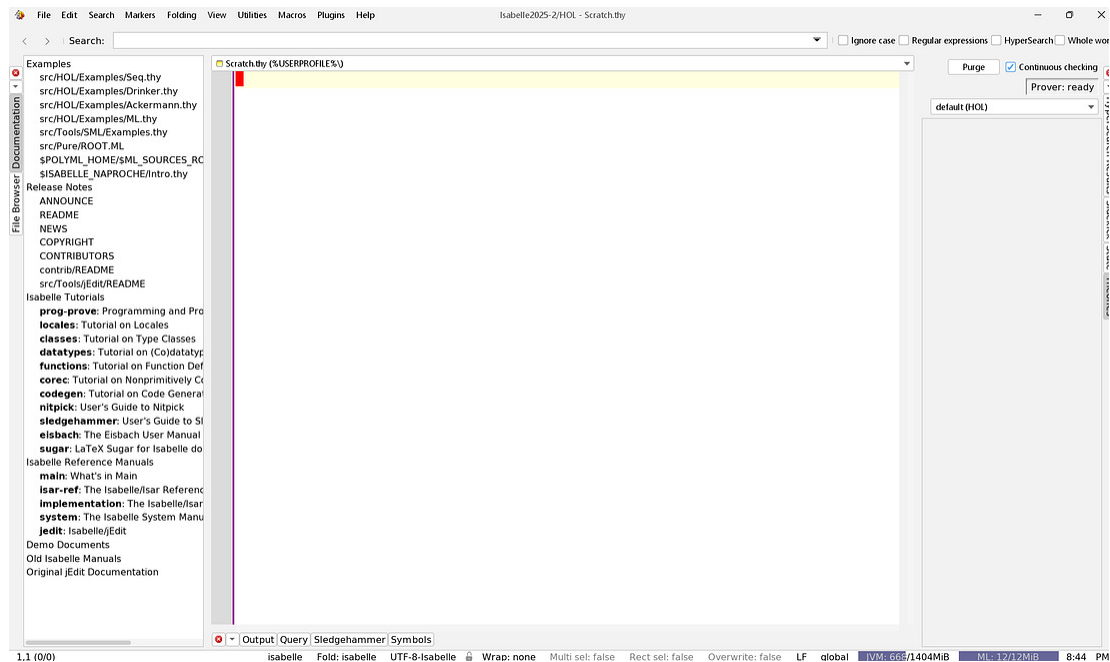
Downloading and installing

So I went to the [official download page](#) and downloaded the .exe file. Apparently, the application lacks developer signatures, so I could not download it using Chrome. I tried with Edge, and it gave me a warning. I then clicked the exe file to unpack it.

Inside the unpacked folder, I clicked the file with the Isabelle icon.

☐ Name	Date modified	Type	Size
 bin	8/15/2026 8:26 PM	File folder	
 contrib	8/15/2026 8:26 PM	File folder	
 doc	8/15/2026 8:27 PM	File folder	
 etc	8/15/2026 8:27 PM	File folder	
 heaps	8/15/2026 8:26 PM	File folder	
 lib	8/15/2026 8:27 PM	File folder	
 src	8/15/2026 8:26 PM	File folder	
 ANNOUNCE	1/17/2026 5:54 PM	File	2 KB
 CONTRIBUTORS	1/17/2026 5:54 PM	File	39 KB
 COPYRIGHT	1/17/2026 5:54 PM	File	2 KB
 Cygwin-Setup	1/18/2026 3:02 PM	Windows Batch File	1 KB
 Cygwin-Terminal	10/7/2025 7:06 AM	Windows Batch File	1 KB
 Isabelle2025-2	1/18/2026 3:02 PM	Application	109 KB
 Isabelle2025-2.exe.manifest	10/7/2025 7:06 AM	MANIFEST File	1 KB
 Isabelle2025-2.l4j	1/18/2026 3:02 PM	Configuration settings	1 KB
 NEWS	1/17/2026 5:54 PM	File	752 KB
 README	1/17/2026 5:54 PM	File	3 KB
 ROOT	1/17/2026 5:54 PM	File	1 KB
 ROOTS	1/17/2026 5:54 PM	File	1 KB

The following image shows the Isabelle interface.



It looks like we have a main text editor and an output panel. So I guess this app may work like TeXStudio.

Creating my first Isabelle file.

Next, I created my first file by clicking on the File, then New. The file was untitled, and I saved it to the correct folder under the name `Xor_Basics.thy` inside my dedicated folder `theories` which is itself inside a folder called `week1`.

The filename **MUST** match the name of the ‘theory’, which will be written after the keyword `theory` exactly!

A *theory* is a self-contained file and logical environment containing a collection of types, function definitions, axioms, and proved theorems. It works like a module, a package, or a class in conventional programming languages.

Here is my ‘theory’ about the commutativity of XOR. The logical symbols are available in Isabelle’s command palette.

```

theory Xor_Basics
  imports Main
begin

(* 1. Define a simple Boolean XOR function *)
definition my_xor :: "bool  $\Rightarrow$  bool  $\Rightarrow$  bool" where
  "my_xor a b  $\equiv$  (a  $\wedge$   $\neg$  b)  $\vee$  ( $\neg$  a  $\wedge$  b)"

(* 2. Test evaluation using the value command *)
value "my_xor True False"
value "my_xor True True"

(* 3. Prove XOR Commutativity *)
lemma xor_comm: "my_xor x y = my_xor y x"
  by (auto simp: my_xor_def)

(* 4. Prove XOR Cancellation Law: (m  $\oplus$  k)  $\oplus$  k = m *)
theorem xor_cancel: "my_xor (my_xor m k) k = m"
  by (auto simp: my_xor_def)

end

```

This code formally defines a custom exclusive OR (XOR) function and automatically proves two of its core logical properties.

What is the comment, and what is the actual code?

(* 1. Define a simple Boolean XOR function *) is a comment. It's not gonna be evaluated. In Isabelle/HOL,

There are three types of Isabelle Comment Style:

- **Standard comments** (*...*) are used for code annotations and explanations. They can span multiple lines and can be nested safely without breaking the syntax. eg: (* 2. Test evaluation using the value command *)
- **Nested comments** are comments inside a comment. They are useful when you are temporarily commenting out blocks of code that already contain comments. eg: (* Outer comment (* Inner comment *) still inside comment *)
- **Formal documentation** is used when you want the comments to appear as formatted text headings or paragraphs in generated PDF reports and proofs. eg: text <4. Prove XOR Cancellation Law: $(m \oplus k) \oplus k = m$ >

Theory Declaration and Setup

theory Xor_Basics is where we declare the name of the theory.

imports Main loads Isabelle's standard library, which allows us to access the type `bool`, the logical connective primitive and the built-in proof engines.

After declaring the module name and calling the library, we need to wrap the description inside a `begin ... end`.

Function Definition & Evaluation

`definition my_xor :: "bool  $\Rightarrow$  bool  $\Rightarrow$  bool"` says that we have a function called `my_xor` that takes as input two `bool` variables and return a `bool` output.

The symbol $\equiv$ assigns the function the standard disjunctive normal formula for XOR $(a \wedge \neg b) \vee (\neg a \wedge b)$.

`value "my_xor True False"` and `value "my_xor True True"` are interactive commands that run Isabelle's code generator to evaluate expressions on the fly. When you put your cursor in the line, their results show next to the cursor in the output window.

```

theory Xor_Basics
  imports Main
begin

(* 1. Define a simple Boolean XOR function *)
definition my_xor :: "bool  $\Rightarrow$  bool  $\Rightarrow$  bool" where
  "my_xor a b  $\equiv$  (a  $\wedge$   $\neg$  b)  $\vee$  ( $\neg$  a  $\wedge$  b)"

(* 2. Test evaluation using the value command *)
value "my_xor True False"
value "my_xor True True"
value "my_xor False True"
value "my_xor False False"

(* 3. Prove XOR Commutativity *)
lemma xor_comm: "my_xor x y = my_xor y x"
  by (auto simp: my_xor_def)

(* 4. Prove XOR Cancellation Law: (m  $\oplus$  k)  $\oplus$  k = m *)
theorem xor_cancel: "my_xor (my_xor m k) k = m"
  by (auto simp: my_xor_def)

end

```



```

theory Xor_Basics
  imports Main
begin

(* 1. Define a simple Boolean XOR function *)
definition my_xor :: "bool  $\Rightarrow$  bool  $\Rightarrow$  bool" where
  "my_xor a b  $\equiv$  (a  $\wedge$   $\neg$  b)  $\vee$  ( $\neg$  a  $\wedge$  b)"

(* 2. Test evaluation using the value command *)
value "my_xor True False"
value "my_xor True True"

(* 3. Prove XOR Commutativity *)
lemma xor_comm: "my_xor x y = my_xor y x"
  by (auto simp: my_xor_def)

(* 4. Prove XOR Cancellation Law: (m  $\oplus$  k)  $\oplus$  k = m *)
theorem xor_cancel: "my_xor (my_xor m k) k = m"
  by (auto simp: my_xor_def)

end

```

Formal Proofs & Automation

- lemma xor_comm: "my_xor x y = my_xor y x": States the mathematical claim that XOR is commutative ($x \oplus y = y \oplus x$).
- by (auto simp: my_xor_def): Instructs Isabelle to expand my_xor_def (the explicit definition of my_xor, created automatically) and use auto (case analysis + simplification) to prove the claim.
- theorem xor_cancel: "my_xor (my_xor m k) k = m": States the main cancellation property: XORing a message m with key k twice yields m.
- by (auto simp: my_xor_def): Proves the cancellation theorem automatically.

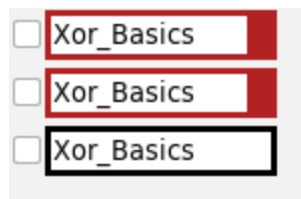
In mathematics, a lemma is a small helper statement, and a theorem is a major result.

In Isabelle, lemma and theorem are functionally identical keywords; using theorem is purely a stylistic choice for significant results. It makes your theory more 'mathy'!

Isabelle uses live visual indicators and proof state tracking to show your code's status instantly:

- Background Text Highlights:
 - **Light Blue / Purple:** Isabelle is actively processing that line in the background.
 - **Pink / Red:** Syntax error, undefined variable, or a tactic failure. Check the Output panel to see the exact error message.
 - **No Highlight (White/Normal):** The line compiled cleanly and passed all verification checks.

In the following screenshot, the first two codes had errors, while the last one passed all verification checks.



- Checking Proof Status:
 - Click on a lemma, theorem, or by command line.
 - If the Output panel shows no remaining subgoals (or goes blank without red text), Isabelle's kernel has successfully checked and accepted the proof.
 - If it shows 1 subgoal..., the proof is incomplete, meaning Isabelle needs additional steps or tactics to complete the

verification of the claim.

8/16/2026

6:24 AM

Creating a second file

It turned out you can prove two theories at once in a single session! I was confused about why we should do that, so I asked Gemini!

Begin Gemini Answer

On-the-fly verification is designed for interactive authoring, whereas sessions are designed for project management, performance, and total system validation. Relying solely on real-time editor checking leaves your work vulnerable to silent logical breaks, massive performance slowdowns, and broken dependencies.

1. Pre-Compiled Caching (Build Performance)

Isabelle's standard library (HOL) and external libraries like the Archive of Formal Proofs (AFP) contain hundreds of thousands of verified theorems. If Isabelle re-proved everything on the fly from raw source files every time you launched the editor, starting your workspace would take 20+ minutes. A session packages verified theory files into a pre-compiled binary snapshot (a "heap image"), allowing you to load massive foundations instantly.

2. Whole-Project Integrity & Hidden Breaks

In Isabelle/jEdit, real-time verification only checks files currently open in your active editor tabs. If you change a core definition in `Xor_Basics.thy`, any closed theory file relying on that definition (like `Modular_Shifts.thy`) could break without triggering a visual error in your open window.

Running a session build forces Isabelle to systematically check every single file in the project dependency tree to guarantee overall logical consistency.

3. Headless Automation & CI/CD

On-the-fly verification requires a graphical user interface (Isabelle/jEdit) with allocated display resources. Automated grading environments, remote servers, and continuous integration pipelines (e.g., GitHub Actions) operate headlessly without a GUI. Session commands (`isabelle build`) allow terminal scripts to non-interactively compile and verify your entire codebase.

4. Library Reusability

Defining a session turns your raw theory files into a structured library. Future projects—or other engineers—can import your entire verified workspace with a single dependency declaration (`session Project2 = Week1 + ...`) without having to manually locate, order, and load your underlying `.thy` source files.

Think of **on-the-fly verification** as your IDE's real-time linter checking your code as you type, and a **session build** as the actual compiler engine (`cargo build`, `npm run build`, or `make`) running in a clean environment to ensure the final product is flawless.

End Gemini Answer

The second file is for modular shift, aka **Caesar cipher** when $n = 26$.

```
theory Modular_Shifts
  imports Main
begin
```

```
(* Define encryption function:  $E(m, k) = (m + k) \bmod n$  *)
```

```

definition E :: "nat ⇒ nat ⇒ nat ⇒ nat" where
  "E n k m ≡ (m + k) mod n"

(* Test encryption on test values *)
value "E 26 3 5"

(* Prove shift composition law *)
theorem shift_compose: "E n k2 (E n k1 m) mod n = E n (k1 + k2) m mod n"
  by (simp add: E_def mod_simps algebra_simps)

end

```

Can you guess what the file name must be?

If you guessed `Modular_Shifts.thy` then you'll be correct! Yay!

Again, we are declaring the theory called `Modular_Shifts` and importing the standard library.

We define a function `E` which takes three natural numbers of type `nat` and outputs a natural number. `mod` is the syntax for the integer modulo arithmetic.

`by (simp add: E_def mod_simps algebra_simps)`: Proves the theorem by expanding `E_def`, applying modulo distribution laws (`mod_simps`), and rearranging addition (`algebra_simps`).

Creating the ROOT Session File

To let Isabelle compile my whole project at once from a command line, I need a plain text ROOT file. This file is located in the `Week1` folder alongside the `theories` folder and has no extension. The filename is just `ROOT`.

```
session "Week1" = HOL +  
  directories  
    "theories"  
  theories  
    "theories/Xor_Basics"  
    "theories/Modular_Shifts"
```

To define a session, you need to declare the list of directories and the list of theories.

Isabelle enforces strict directory isolation for sessions. When you list "theories/Xor_Basics" without telling Isabelle that "theories" is an approved session directory, it flags an implicit directory error to prevent accidental file inclusion outside your project bounds. Adding directories "theories" grants explicit permission.

Note: Session names don't need to match folder names; theory names must match their exact filenames.

Testing the Build (The “Compilation” Step)

My Isabelle folder is at D:\Repositories\She Crypts\Isabelle2025-2. I do not want to add the filename to my Windows path because I use an external drive.

To compile my theories, I open File Explorer and go to my Isabelle installation folder: D:\Repositories\She Crypts\Isabelle2025-2\

I double-clicked the file named Cygwin-Terminal.bat and pasted

```
cd "/cygdrive/d/Repositories/She Crypts/formal/week1"
```

Then run the command

```
isabelle build -D .
```

This is the output:

```
$ isabelle build -D .  
Running Week1 ...  
Finished Week1 (0:00:08 elapsed time)  
0:00:18 elapsed time
```

“Finished Week1” means my mathematical claims are now 100% mechanically verified for every possible input in existence.

Traditional software testing uses KAT (Known Answer tests), which verify properties for multiple specific numbers. Isabelle has just mathematically proved that the cancellation and composition laws in modular shifts hold true across *infinite* possible values of m , k , and n .

Isabelle’s LCF-style logical kernel independently checked every single logical deduction step behind my proofs. It did not rely on hand-waving or unproven assumptions. This means that now I have **Kernel-Level Soundness**.

My project built cleanly without sorry keywords (Isabelle’s placeholder for incomplete proofs), circular logic, or unfulfilled premises. This means I can be confident I have Zero Hidden Errors.

My build environment is fully configured. Anyone who pulls my repository and runs `isabelle build -D .` will get the exact same zero-error result.

Putting the result into LaTeX.

Isabelle uses a built-in **Document Preparation System**. I do not retype my code by hand in LaTeX; instead, I write explanatory text directly in my theory files, and Isabelle automatically compiles my verified code into a professional LaTeX PDF.

For this to work, I need a local **MiKTeX** installation with LuaLaTeX. If you install MikTeX after opening the Cygwin terminal, you need to restart the terminal.

To enable LaTeX output, I need to create a `document/` folder with a tex file `root.tex` alongside my `theories/` folder:

```
week1/
├── ROOT
├── theories/
│   ├── Xor_Basics.thy
│   └── Modular_Shifts.thy
└── document/
    └── root.tex
```

This is the content of `root.tex`

```
\documentclass[11pt,a4paper]{article}
\usepackage{isabelle,isabellesym}

\title{Week 1 Verification Report}
\author{She Crypts Team}

\begin{document}
\maketitle

% Isabelle automatically replaces this line with all your .thy files
\input{session}
```



```
\end{document}
```

Then I need to update the ROOT file to instruct Isabelle to build the PDF.

```
session "Week1" = HOL +  
  options [document = pdf, document_output = "output"]  
  directories  
    "theories"  
  theories  
    "theories/Xor_Basics"  
    "theories/Modular_Shifts"  
  document_files  
    "root.tex"
```

I also need to change my files to have text documentation instead of standard comments

After that, I went back to the same Cygwin terminal and reran

```
isabelle build -D .
```

This is the terminal output:

```
$ isabelle build -D .  
Running Week1 ...  
Preparing Week1/document ...  
Finished Week1/document (0:00:06 elapsed time)  
Document at "/cygdrive/d/Repositories/She Crypts/formal/week1/output/document.pdf"  
Finished Week1 (0:00:07 elapsed time)  
0:00:26 elapsed time
```

You can view the PDF [here](#).

That's it for week 1. Talk to you next week!

Thanks for reading shecrypts Publication!
Subscribe for free to receive new posts and
support my work.

✓ **Subscribed**